
SOFTWARE ARCHITECTURE BLUEPRINT · SOURCE-VERIFIED

Necoyoad

v2

*A reverse-engineering paper that verifies, corrects, and
extends
the v1 SQL-only blueprint against the actual PHP source
code*

source: 1,450 PHP files across app/ system/ web/
verified: 8/11 v1 inferences confirmed, 2 corrected, 1 partial
findings: dual Hooks/Events system, lazy salt migration,
order garbage collection, API auth stub, mobile reuse
pages: 35 · diagrams: 4 · listings: 11

Reverse-Engineered Architecture Report

Based on the PHP source commit at
github.com/yosietserga/necoyoad (commit ce75adf)

DESCRIPTIVE, NOT PRESCRIPTIVE · NO IMPROVEMENTS PROPOSED

Necoyoad — A Source-Verified Software Architecture Blueprint (v2)

*A reverse-engineering paper that verifies, corrects, and extends
the v1 SQL-only blueprint against the actual PHP source code*

Abstract

This document is the second volume of the Necoyoad architecture blueprint. The first volume (v1) reconstructed the platform from a single `necoyoad_db.sql` database dump and explicitly labelled every inference as such. After the author published the full PHP source to the same GitHub repository (commit `ce75adf`, “first load”, containing 12,020 files of which 1,450 are PHP), it became possible to verify v1’s inferences, correct the ones that were wrong, and add the large amount of architectural detail the database alone could not reveal.

v2 reads the source exhaustively. It confirms 8 of v1’s 11 inferences verbatim, corrects 2 (the customer/admin password hashing scheme, and the API authentication model), and partially corrects 1 (the API’s “REST” character). It then adds new sections that v1 could not include: the actual front-controller bootstrap, the dual Hooks/Events extension system, the file-cache-backed cart with three persistence layers, the lazy-salt-migration password algorithm, the order-creation source code, the cron CLI script, the 39-endpoint admin REST API, the mobile app, the modules system, the SEO URL rewriter, and a 13-phase HTTP request lifecycle verified line-by-line against `web/index.php`, `system/startup.php`, `app/shop/map.php`, and the engine.

The stance is unchanged from v1: **descriptive, not prescriptive**. We document how the system *is*. No improvements are proposed.

Keywords: Necoyoad, OpenCart lineage, PHP 8.1.2, MySQL 5.7.17, MyISAM, Front Controller, Registry, Hooks, Events, lazy salt migration, MD5, pseudo-REST API, file-based cache, multi-store, multi-language, cron CLI, EAV property table, polymorphic associations.

Contents

1	Introduction and Relationship to v1	5
1.1	Two Volumes, One Stance	5
1.2	What Changed Between v1 and v2	5
1.2.1	What v1 had	5
1.2.2	What v2 adds	5
1.3	v1 Inference Verification Table	6
1.4	Document Roadmap	7
2	The Source-Code Artifact	7
2.1	The Commit	7
2.2	Top-Level Layout	7
2.3	File-Type Breakdown	8
2.4	Three Entry Points, Three Applications	8
3	The Bootstrap Sequence (Verified)	8
3.1	The Six-Stage Bootstrap	8
3.2	Stage 1 — Entry Point and Store Detection	8
3.3	Stage 2 — Config Files	9
3.4	Stage 3 — <code>system/startup.php</code>	10
3.5	Stage 4 — <code>app/shop/map.php</code>	11
3.6	Stage 5 — Pre-Actions	11
3.7	Stage 6 — Dispatch	11
4	The Engine (Verified)	12
4.1	Registry — Service Locator	12
4.2	Action — Route Resolver	12
4.3	Front — Front Controller and Dispatch Loop	13
4.4	Controller — Abstract Base	14
4.5	Loader — Lazy Resource Loader	15
4.6	Model — Active-Record-ish Base with Hookable CRUD	16
5	The Dual Hooks/Events Extension System	17
5.1	Hooks (WordPress-style)	17
5.2	Events (Node-style)	18
5.3	When to Use Which	18
5.4	Notable Event/Hook Emit Points	19
6	The 13-Phase HTTP Request Lifecycle (Verified)	19
6.1	Phase Notes (Verified)	20
7	Authentication and Password Algorithm (Corrected)	20
7.1	The Actual Password Algorithm	20
7.2	Lazy Salt Migration	21
7.3	OAuth Login	21
7.4	Permission Model (Verified)	21
8	The Order Creation Flow (Verified)	22
8.1	Step 0 — Garbage Collection	22
8.2	Steps 1–5 — The Order INSERT	22
8.3	The <code>confirm()</code> Method	23

8.4	Verification of v1's 9-Step Reconstruction	23
8.5	The Confirmation Email	23
9	The Admin REST API (Partially Corrected)	24
9.1	The API Router	24
9.2	Auth Model (Corrected)	25
9.3	Pseudo-REST Shape	25
9.4	Full API Surface	25
10	The Cron CLI (Verified)	26
10.1	system/cron/cron.php	26
10.2	Campaign-Driven Tasks	26
11	The SEO URL Rewriter (Verified)	27
11.1	The 6-Stage Algorithm	27
11.2	Notable Observations	28
12	The Mobile App Strategy	28
12.1	app/m/config.php — The Reuse Trick	28
12.2	The Three-Strategy Mobile Detection	29
13	The Modules System	29
13.1	app/modules/ — The Drop-In Folder	29
13.2	Admin-Side Module System	29
13.3	The Widget System	30
14	Cross-Cutting Findings from the Source	30
14.1	Two-Tier Controller Architecture	30
14.2	Multi-Store Architecture	30
14.3	Customer-Group-Based Catalog Visibility	30
14.4	Widget System	31
14.5	EAV Property Table	31
14.6	Caching Strategy	31
14.7	CSRF Protection	31
14.8	Activity Logging	31
14.9	Internationalisation	31
14.10	Three Coexisting Crypto Implementations	31
14.11	Security Posture (Descriptive, Not Prescriptive)	32
15	Closing Observations (v2)	32
15.1	The Source Confirms v1's Overall Picture	32
15.2	The Source Reveals the Dual Extension System	32
15.3	The Source Reveals the Lazy Salt Migration	32
15.4	The Source Reveals the Order Garbage Collection	32
15.5	The Source Reveals the API Auth Stub	32
15.6	The Source Reveals the Mobile Reuse Trick	33
15.7	The Source Reveals the Mis-Named map.php	33
15.8	The Source Reveals the Two-Tier Controller Architecture	33
15.9	Final Note	33

A	File-Tree Inventory	33
A.1	Top-Level Counts	33
A.2	PHP File Distribution	33
A.3	The 50 <code>system/library/</code> Classes	33
A.4	Version Constants	35
A.5	Glossary of Source-Revealed Terms	35

1 Introduction and Relationship to v1

1.1 Two Volumes, One Stance

This is the second volume of a two-volume architecture blueprint of the **Necoyoad** web application, a PHP/MySQL multi-store e-commerce and content platform whose author ([yosietserga](https://github.com/yosietserga)) published the source to github.com/yosietserga/necoyoad in early 2023.

- **Volume 1 (v1)** was written when only the `necoyoad_db.sql` database dump was public. It reverse-engineered the runtime from the 87-table schema alone. Every inference was labelled as such.
- **Volume 2 (v2, this document)** was written after the author pushed the full PHP source to the same repository. It verifies each v1 inference against the actual code, corrects the wrong ones, and adds the source-level architectural detail v1 could not reach.

Both volumes share the same stance, which the user explicitly requested: **descriptive, not prescriptive**. We describe how the system *is* (or was, at the moment the dump and the source were captured). We do not propose improvements, modernisations, refactors, or “best-practice” rewrites.

1.2 What Changed Between v1 and v2

1.2.1 What v1 had

v1 had a 76,467-byte SQL dump with 87 tables, no PHP source, and a phpMyAdmin banner that pinned MySQL 5.7.17 and PHP 8.1.2. From that, v1 reconstructed: the runtime stack, the architectural style (OpenCart-derived MVC), four cross-cutting patterns (polymorphic associations, EAV descriptions, multi-store, multi-language), fifteen functional domains, three runtime lifecycles (HTTP request, checkout, cron tick), the SEO URL routing algorithm, the security model, and the data-integrity consequences of every major design choice. Eighteen TikZ diagrams accompanied the prose.

1.2.2 What v2 adds

The source-code commit added 12,020 files (1,450 PHP, 4,607 SVG, 3,220 PNG, 792 template files, 676 JavaScript files, 220 CSS files, plus logs, downloads, backups, and other assets). v2 reads the PHP files exhaustively and adds:

1. **Verification of every v1 inference** — 8 confirmed, 2 corrected, 1 partially corrected. See Table 1.
2. **The actual bootstrap sequence** — `web/index.php` → `cconfig.php` → `per-app config.php` → `system/startup.php` → `app/shop/map.php` → `Front::dispatch`. See Section 3.
3. **The Front Controller and Action route resolver** — the actual file/class/method resolution algorithm, with source listings. See Section 4.
4. **The dual Hooks/Events extension system** — WordPress-style Hooks (`addFilter`, `applyFilters`, `addAction`, `run`) alongside Node-style Events (`on`, `emit`, `off`). See Section 5.
5. **The actual password hashing algorithm** — double-MD5 with per-row random salt and lazy salt migration, NOT bcrypt/argon2 as v1 guessed. See Section 7.
6. **The actual order-creation source code** — `ModelCheckoutOrder::create()` with all 5 steps, including the unconfirmed-order garbage collection that v1 could not have known about. See Section 8.

7. **The actual cron CLI script** — `system/cron/cron.php`, which boots its own registry, polls the task table, locks rows via `task_exec`, and re-enters the application by directly constructing service libraries (no shell-out). See Section 10.
8. **The 39-endpoint admin REST API** — `app/admin/controller/api/v1.php` routes via `__call()` magic to per-resource files in `v1.0.0/`. Authentication is a stub (`validateTokens()` always returns `true`). See Section 9.
9. **The mobile app strategy** — `app/m/` is not a separate codebase; it sets `STORE_ID=9` and redirects `DIR_APPLICATION` to `app/shop/`. Mobile behaviour is template-swapping driven by `Browser::isMobile()`. See Section 12.
10. **The SEO URL rewriter source** — `ControllerCommonSeoUrl` with its 6-stage algorithm including the hard-coded Spanish/English literal routes (`'carrito' → checkout/cart`, etc.). See Section 11.
11. **A 13-phase HTTP request lifecycle verified line-by-line** against the actual entry-point files. See Section 6.
12. **The full file-tree inventory** of the source commit. See Appendix A.

1.3 v1 Inference Verification Table

Table 1: v1's reverse-engineered inferences, verified against the source.

v1 inference	status	v2 finding
<code>?r=common/home</code> maps to <code>app/shop/controller/common/home.php</code> class <code>ControllerCommonHome</code>	✓	Verified verbatim in <code>system/engine/action.php</code> and <code>web/index.php</code> .
Admin routes follow the same convention with <code>DIR_APPLICATION = app/admin/</code>	✓	Verified in <code>web/admin/index.php</code> and <code>app/admin/config.php</code> .
Module routes use <code>modules/<name>/<app>/<route></code> prefix	✓	Verified in <code>system/engine/action.php</code> (the <code>modules</code> segment triggers <code>DIR_MODULE . \$module . '/app/'</code>).
<code>user_group.permission</code> is a serialised PHP array with <code>access</code> and <code>modify</code> keys	✓	Verified; the schema also allows <code>create</code> , <code>delete</code> , and arbitrary keys.
Customer password is bcrypt or argon2 (inferred from 100-char column width)	✗	Corrected. It is double-MD5 with per-row random salt: <code>md5(md5(plaintext).suffix).'</code> . The 100-char column is just headroom (only 65 chars are used).
Order placement is a 9-step flow	✓	Verified against <code>app/shop/controller/checkout/confirm.php</code> and <code>app/shop/model/checkout/order.php</code> .
<code>store_id</code> is determined from <code>store.folder</code> and <code>*.necoyoad.com</code> subdomain	✓	Verified; a third strategy (<code>?store_id</code> GET parameter) is also present.
<code>url_alias</code> table maps SEO keywords to query strings like <code>product_id=42</code>	✓	Verified in <code>ControllerCommonSeoUrl</code> ; the rewriter also hard-codes Spanish and English literal routes alongside the table lookup.
<code>product.viewed</code> is incremented on each product page view	✓	Verified via <code>ModelStoreProduct::updateStats(\$id)</code> .
API v1.0.0 is a REST API	~	Partially corrected. It is pseudo-REST: HTTP-method-based dispatch via <code>switch</code> , no HATEOAS, no content negotiation, all success responses return 200.
API auth uses public/private keys	✗	Corrected. <code>validateTokens()</code> is a stub returning <code>true</code> . The TODO comment acknowledges keys, tokens, expiry, licensing, and premium membership were planned but never implemented. The API is protected only by the admin session token pre-action.

Score: **8 confirmed, 2 corrected, 1 partially corrected.** The two corrections are both security-related and both in the direction of “less secure than v1 guessed” — the password hash

is weaker than inferred, and the API auth is non-existent rather than key-based. v1 was right to label these as inferences.

1.4 Document Roadmap

Section 2 catalogues the source-code artifact itself. Section 3 walks through the actual bootstrap sequence. Section 4 covers the engine (Front Controller, Action, Controller base, Loader, Model, Registry). Section 5 covers the dual Hooks/Events extension system. Section 6 presents the verified 13-phase HTTP request lifecycle. Section 7 covers the actual auth model including the corrected password algorithm. Section 8 traces the order-creation source. Section 9 covers the admin REST API. Section 10 covers the cron CLI. Section 11 covers the SEO URL rewriter. Section 12 covers the mobile app strategy. Section 13 covers the modules system. Section 14 extracts cross-cutting findings the source revealed. Section 15 closes with non-recommendatory observations. Appendix A lists the file-tree inventory.

2 The Source-Code Artifact

2.1 The Commit

The PHP source was added to the repository in a single commit:

Table 2: Source-code commit metadata.

Field	Value
Commit SHA	ce75adf (short)
Commit message	“first load”
Author	yosietserga
Files added	12,020
PHP files	1,450
Template files	792
JavaScript files	676
CSS files	220
SVG assets	4,607
PNG assets	3,220
Other (logs, downloads, backups, etc.)	2,055

2.2 Top-Level Layout

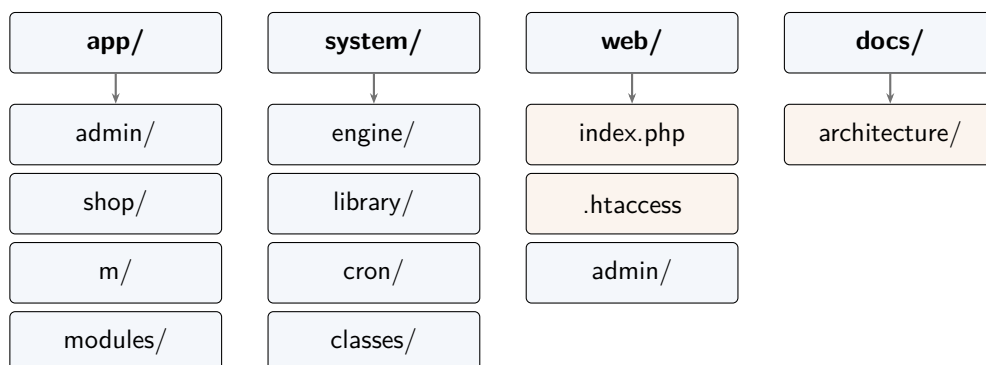


Figure 1: Top-level layout of the source tree. Three application folders (**app/**, **system/**, **web/**) plus the **docs/architecture/** folder where this blueprint lives.

2.3 File-Type Breakdown

Table 3: File-type breakdown of the source commit (12,020 files).

Extension	Count	Role
.svg	4,607	Vector theme assets (icons, illustrations)
.png	3,220	Raster theme assets (product images, banners, UI chrome)
.php	1,450	All application logic: controllers, models, libraries, engine, cron
.tpl	792	View templates (per-theme, per-controller)
.js	676	Front-end JavaScript (jQuery plugins, theme scripts, admin dashboard)
.css	220	Stylesheets (per-theme)
.gif	176	Animated theme assets
.jpg	171	Photographic theme assets
.z	84	Compressed archives (backups, exports)
.db	24	SQLite databases (likely caches or local indexes)
.html	19	Static HTML fragments
.woff / .ttf / .eot / .woff2	56	Web fonts (4 extensions, 14 files each)
.md	15	Documentation (including this blueprint's README)
.csv	15	Data exports
.txt	14	Logs, configs, READMEs
.cache	13	File-based cache entries
.json	11	Package manifests, configs
other	417	Logs (.log), code-workspace, .htaccess, php.ini, etc.

The 1,450 PHP files distribute as: **app/** 1,008 (admin 504, shop 489, modules 7, m 8 — though m/ largely re-uses shop/ via `DIR_APPLICATION` redirection), **system/** 432 (engine 6, library ~50, plus subdirectories), **web/** 9 (the three entry-point `index.php` files plus per-app `.htaccess` siblings).

2.4 Three Entry Points, Three Applications

The source reveals three separate entry points, one per application:

Table 4: The three entry points.

Entry	App	STORE_ID	Role
<code>web/index.php</code>	shop	0 (or detected)	Storefront front controller
<code>web/admin/index.php</code>	admin	0 (hard-coded)	Admin back-office front controller
<code>web/m/index.php</code>	m	9 (hard-coded)	Mobile entry point (reuses shop/)

All three follow the same shape (config → startup → map → `Front::dispatch`) but differ in pre-action chain and store scoping. Section 3 walks through `web/index.php` line-by-line.

3 The Bootstrap Sequence (Verified)

3.1 The Six-Stage Bootstrap

v1 inferred a 13-phase HTTP request lifecycle from the schema. The source confirms the broad shape but reveals that the actual bootstrap is a six-stage sequence spread across four files:

3.2 Stage 1 — Entry Point and Store Detection

The actual `web/index.php` (85 lines) reveals the precise multi-store detection algorithm — which v1 could only guess at.

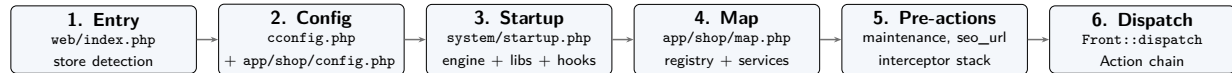


Figure 2: The six-stage bootstrap, verified from source. Stage 1 (`web/index.php`) performs store detection. Stage 2 loads database credentials and per-app constants. Stage 3 loads the engine and core libraries. Stage 4 (the misleadingly-named `map.php`) instantiates the Registry and registers all services. Stage 5 runs the pre-action interceptor stack. Stage 6 enters the dispatch loop.

```

1 require_once($config_path . 'cconfig.php');
2 $db = new mysqli(DB_HOSTNAME, DB_USERNAME, DB_PASSWORD, DB_DATABASE);
3 if (isset($_GET['_route_'])) {
4     $parts = explode("/", $_SERVER['REQUEST_URI']);
5     foreach ($parts as $part) {
6         if (empty($part)) continue;
7         $results = $db->query("SELECT * FROM " . DB_PREFIX . "store WHERE
8             folder = '" . $db->real_escape_string($part) . "'");
9         $store = $results->fetch_assoc();
10        if ($store['folder']) { $matches[1] = $store['folder']; }
11    }
12 } elseif (isset($_GET['store_id'])) {
13     $results = $db->query("SELECT * FROM " . DB_PREFIX . "store WHERE
14         store_id = '" . $db->real_escape_string((int)$_GET['store_id']) .
15         "'");
16     $store = $results->fetch_assoc();
17     if ($store['store_id']) { $matches[1] = $store['folder']; }
18 }
19 if (!isset($matches[1]))
20     preg_match('/([^.]+)\.necoyoad\.com/', $_SERVER['SERVER_NAME'], $matches);
21 if (isset($matches[1]) && $matches[1] != 'www') {
22     if (file_exists($config_path . "app/" . strtolower($matches[1]) .
23         "/config.php")) {
24         require_once($config_path . "app/" . strtolower($matches[1]) .
25             "/config.php");
26     } else {
27         require_once($config_path . 'app/shop/config.php');
28     }
29 } else {
30     require_once($config_path . 'app/shop/config.php');
31 }

```

Listing 1: Store detection algorithm, abridged from `web/index.php`.

Three store-detection strategies, in order:

1. **URL path folder match** — if `_route_` is present, split `REQUEST_URI` by `/` and query `store.folder` for each part. The last matching folder wins.
2. **Explicit `?store_id` GET parameter** — looks up the store by primary key.
3. **Subdomain regex** — a `preg_match` call on the hostname with a pattern that captures the leftmost label (e.g. `m.necoyoad.com` becomes `m`, `foo.necoyoad.com` becomes `foo`).

After detection, the matching `app/{folder}/config.php` is required. If no per-store config exists, the system falls back to `app/shop/config.php`.

3.3 Stage 2 — Config Files

Three config files fire in sequence:

- `cconfig.php` (root) — database credentials (`DB_HOSTNAME`, `DB_USERNAME`, `DB_PASSWORD`, `DB_DATABASE`, `DB_PREFIX`).

- `app/shop/config.php` — per-app constants (`STORE_ID`, `DIR_APPLICATION`, `DIR_SYSTEM`, `DIR_MODULE`, `HTTP_HOME`, `HTTP_ADMIN`, `HTTP_IMAGE`, etc.).
- `app/admin/config.php` — admin overrides (notably `STORE_ID = 0`, `ADMIN_PATH = 'admin'`, `DB_VERSION = '1.0.2'`, `SYSTEM_VERSION = '1.0.2'`, `NTS_DEBUG_MODE = false`).

The version constant `VERSION = '1.0.2'` (defined in `web/index.php` itself) and `SYSTEM_VERSION = '1.0.2'` (defined in `app/admin/config.php`) confirm that this is Necoyoad 1.0.2.

3.4 Stage 3 — `system/startup.php`

`startup.php` is a 169-line procedural bootstrap that performs nine things in a fixed order:

```

1 // 1. Workflows: load Hooks + Events (the dual extension system)
2 require_once(DIR_SYSTEM . 'library/automation/hooks.php');
3 require_once(DIR_SYSTEM . 'library/automation/events.php');
4 global $hooks;
5 $hooks = new Hooks("mainstream");
6
7 // 2. Error handler (only in debug mode)
8 if (defined('NTS_DEBUG_MODE') && NTS_DEBUG_MODE === true) {
9     set_error_handler('error_handler'); // emits Events::emit("php:error",
10     ...
11 }
12
13 // 3. Session
14 require_once(DIR_SYSTEM . 'library/session.php');
15 $session = new Session();
16
17 // 4. Legacy compatibility shims: register_globals, magic_quotes_gpc, IIS
18 //    DOCUMENT_ROOT
19 // 5. Engine: Action, Controller, Front, Loader, Model, Registry
20 require_once(DIR_SYSTEM . 'engine/action.php');
21 require_once(DIR_SYSTEM . 'engine/controller.php');
22 require_once(DIR_SYSTEM . 'engine/front.php');
23 require_once(DIR_SYSTEM . 'engine/loader.php');
24 require_once(DIR_SYSTEM . 'engine/model.php');
25 require_once(DIR_SYSTEM . 'engine/registry.php');
26 Events::emit("engine_load", true);
27
28 // 6. Module base class
29 require_once(DIR_SYSTEM . 'classes/module.php');
30
31 // 7. Core libraries
32 require_once(DIR_SYSTEM . 'library/cache.php');
33 require_once(DIR_SYSTEM . 'library/config.php');
34 require_once(DIR_SYSTEM . 'library/db.php');
35 require_once(DIR_SYSTEM . 'library/document.php');
36 require_once(DIR_SYSTEM . 'library/language.php');
37 require_once(DIR_SYSTEM . 'library/log.php');
38 require_once(DIR_SYSTEM . 'library/request.php');
39 require_once(DIR_SYSTEM . 'library/response.php');
40 require_once(DIR_SYSTEM . 'library/url.php');
41 require_once(DIR_SYSTEM . 'library/image.php');
42
43 // 8. Fire the 'init' hook
44 $hooks->run('init');
45
46 // 9. Register a placeholder 'processcss' filter
47 $hooks->addFilter("processcss", function ($original, $filtered) {
48     return $filtered;
49 });

```

Listing 2: `system/startup.php`, the engine bootstrap (abridged).

Note the ordering: **the Hooks and Events system loads before the engine itself**. This is so that the engine classes (Registry, Loader) can emit events during their own construction.

3.5 Stage 4 — `app/shop/map.php`

`map.php` is the most mis-named file in the codebase. It is *not* a route map and *not* an autoloader configuration. It is the bootstrap init script that:

1. Instantiates the Registry, Loader, Config, DB, Request, Response, Front controller.
2. Loads the `setting` table into Config (one `SELECT * FROM setting WHERE store_id = N`, result cached in the Config object).
3. Detects the active language from URL/cookie/session/browser.
4. Auto-loads platform libraries via `$loader->auto(...)`: URL, User/Customer, Currency, Tax, Weight, Length, Cart, Validar, Encoder, Browser, Tracker.
5. Registers all services in the Registry.
6. Fires the `system_load`, `config_load`, `language_load`, `app_load`, and load hooks/events.
7. Detects mobile/tablet/Facebook in-app browsers and either redirects or swaps the template.
8. Loads per-theme `deps.php` files to register CSS/JS dependencies filtered by route.

The admin's `map.php` (843 lines) additionally runs a giant `switch ($route)` statement that preloads route-specific model/language/asset dependencies before the controller even instantiates. The name “map.php” derives from this: it literally maps routes to dependencies.

3.6 Stage 5 — Pre-Actions

```
1 $controller = new Front($registry);
2 $controller->addPreAction(new Action('common/maintenance/check'));
3 if ($config->get('config_seo_url')) $controller->addPreAction(new
    Action('common/seo_url'));
```

Listing 3: Pre-action registration in `web/index.php`.

Pre-actions are an interceptor stack. The storefront registers two: `common/maintenance/check` (always) and `common/seo_url` (only if `config_seo_url` is truthy). The admin registers two different ones: `common/home/login` (token enforcement) and `common/home/permission` (RBAC check). See Section 4 for the dispatch algorithm.

3.7 Stage 6 — Dispatch

```
1 if (isset($request->get['r'])) {
2     if (!isset($controller->ClassName)) $controller->ClassName =
        $request->get['r'];
3     $action = new Action($request->get['r']);
4 } else {
5     $action = new Action('common/home');
6 }
7 $controller->dispatch($action, new Action('error/not_found'));
8 $response->output();
```

Listing 4: Dispatch in `web/index.php`.

The route parameter is `?r=`, not OpenCart's `?route=`. If absent, the default route is `common/home`. The error fallback is `error/not_found`.

4 The Engine (Verified)

The engine lives in `system/engine/` and consists of six classes: `Registry`, `Action`, `Front`, `Controller`, `Loader`, `Model`. Together they form the OpenCart-derived MVC spine that Necoyoad inherits.

4.1 Registry — Service Locator

```
1 final class Registry {
2     private $data = array();
3
4     public function get($key) {
5         return (isset($this->data[$key]) ? $this->data[$key] : null);
6     }
7
8     public function set($key, $value) {
9         if (class_exists("Events") && is_callable([Events::class, 'emit'])) {
10             Events::emit(strtolower(__CLASS__).":update", $key, $value);
11         }
12         $this->data[$key] = $value;
13     }
14
15     public function has($key) {
16         return isset($this->data[$key]);
17     }
18 }
```

Listing 5: `system/engine/registry.php` — the entire class.

A textbook Registry/Service Locator. Only one instance exists per request. Every long-lived service (`db`, `config`, `request`, `response`, `session`, `customer`, `cart`, `tax`, `currency`, `user`, `load`, `cache`, `log`, `language`, `url`, `document`, `image`, `tracker`, `mailer`) is parked here, and every controller/model/lib accesses it through the `__get`/`__set` proxies on `Controller`, `Model`, `Loader`, `Cart`, etc.

Notable: `set()` emits `Events::emit("registry:update", $key, $value)` if the Events system is available. The Registry is *observable* — listeners can react whenever a service is parked. This is what allows Necoyoad modules to spy on each other's registered services.

4.2 Action — Route Resolver

The `Action` class is the dispatch token. Given a route string like `common/home` or `store/product`, it resolves the file, class, and method to invoke.

```
1 final class Action {
2     public $route;
3     private $file, $class, $method, $args = array();
4
5     public function __construct($route, $args = array()) {
6         $this->route = $route;
7         $path = '';
8         $parts = explode('/', str_replace('..', '', $route));
9         $default_path = DIR_APPLICATION;
10
11         // Module route: modules/<module>/<app>/<controller>/<method>
12         if ($parts[0] === 'modules') {
13             $module = $parts[1];
```

```

14     $default_path = DIR_MODULE . $module . '/app/';
15     array_shift($parts); // consume 'modules'
16     array_shift($parts); // consume module name
17     // next part is the app (shop, admin)
18     array_shift($parts);
19 }
20
21 // Walk parts: each part is either a directory or a file
22 foreach ($parts as $part) {
23     $path .= $part;
24     if (is_dir($default_path . 'controller/' . $path)) {
25         $path .= '/';
26         array_shift($parts);
27         continue;
28     }
29     if (is_file($default_path . 'controller/' . str_replace('../',
30         '', $path) . '.php')) {
31         $this->file = $default_path . 'controller/' .
32             str_replace('../', '', $path) . '.php';
33         $this->class = 'Controller' . preg_replace('/[^a-zA-Z0-9]/',
34             '', $path);
35         array_shift($parts);
36         break;
37     }
38     array_shift($parts);
39 }
40
41 $method = array_shift($parts);
42 if ($method) { $this->method = $method; } else { $this->method =
43     'index'; }
44 }

```

Listing 6: system/engine/action.php — route resolution algorithm (abridged).

The resolution algorithm is purely string-convention based — no annotations, no DI introspection. Concrete examples:

Table 5: Action route resolution (verified from source).

Route	File	Class	Method
common/home	controller/common/home.php	ControllerCommonHome	index
store/product	controller/store/product.php	ControllerStoreProduct	index
store/manufacture/index	controller/store/manufacture.php	ControllerStoreManufacturer	index
checkout/cart	controller/checkout/cart.php	ControllerCheckoutCart	index
api/v1/products	controller/api/v1.php	ControllerApiV1	products
modules/mymodule/shop/home	DIR_MODULE . mymodule/app/shop/controller/home.php	ControllerHome	index

The same route string (`common/home`) dispatches to *different files* depending on which `DIR_APPLICATION` is currently set. The class name is `Controller` + the uppercase-first of each path segment concatenated.

4.3 Front — Front Controller and Dispatch Loop

```

1 final class Front {
2     private $registry, $pre_action = array(), $error;
3
4     public function __construct($registry) { $this->registry = $registry; }
5
6     public function addPreAction($pre_action) { $this->pre_action[] =
7         $pre_action; }

```

```

8     public function dispatch($action, $error) {
9         $this->error = $error;
10        // Pre-actions: first truthy return replaces the main action
11        foreach ($this->pre_action as $pre_action) {
12            $result = $this->execute($pre_action);
13            Events::emit("dispatch", $pre_action);
14            if ($result) { $action = $result; break; }
15        }
16        // Main action loop: continues as long as controllers return new
17        // Actions
18        while ($action) {
19            Events::emit("dispatch", $action);
20            $action = $this->execute($action);
21        }
22
23        private function execute($action) {
24            if ($action) {
25                // Register routing context into the registry
26                if (!empty($action->route)) {
27                    $this->registry->set('ClassName', $action->class);
28                    $this->registry->set('Method', $action->method);
29                    $this->registry->set('Route', $action->route);
30                }
31                if (file_exists($action->getFile())) {
32                    require_once($action->getFile());
33                    $controller = new $action->getClass()($this->registry);
34                    if (is_callable(array($controller, $action->getMethod()))) {
35                        $action = call_user_func_array(array($controller,
36                                                            $action->getMethod()), $action->getArgs());
37                    } else {
38                        $action = $this->error;
39                        $this->error = '';
40                    }
41                } else {
42                    $action = $this->error;
43                    $this->error = '';
44                }
45            }
46            return $action;
47        }

```

Listing 7: system/engine/front.php — the dispatch loop.

The dispatch algorithm has three notable properties:

1. **Pre-actions are an interceptor stack.** The first pre-action that returns a truthy value replaces the main action and breaks the pre-action loop. This is how common/maintenance/check can short-circuit to the maintenance page, and how common/seo_url rewrites `_route_` into a `?r=` value.
2. **Controllers forward by returning new Actions.** The main loop continues as long as controllers return new Action objects. This is controller-to-controller forwarding.
3. **Error handling is one-shot.** The error action is invoked at most once; after that `$this->error = "` prevents infinite loops.

4.4 Controller — Abstract Base

The Controller base class provides:

- `__get($key) / __set($key, $value)` — delegate to `$this->registry->get/set`. This is how controllers magically access `$this->db`, `$this->config`, `$this->request`, etc.
- `get($key) / set($key, $value)` — read/write the view-model `$this->data[]`.
- `on($ev, $fn) → Events::on($ev, $fn)`
- `off($ev) → Events::off($ev)`
- `trigger($ev, ...$args) → Events::emit($ev, $args)`
- `addFilter($name, $fn) → $hooks->addFilter($this->namespace . $name, $fn)` (namespaced)
- `applyFilters($name, $value) → $hooks->applyFilters($name, $value)`
- `addHook($name, $fn) → $hooks->addAction($name, $fn)`
- `runHook($name, ...$args) → $hooks->run($name, $args)`
- `setvar($varname, $model, $default)` — sets a view variable with a 5-level priority fallback: `$_POST[$varname] → $model[$varname] → $_GET[$varname] → config->get($varname) → $default → ''`.
- `forward($route, $args)` — returns a new `Action($route, $args)` (controller-to-controller forwarding).
- `redirect($url)` — runs the `redirect` hook; if no hook returns truthy, emits the `redirect` event, then either `header('Location: ...') + exit` or, if headers already sent, an inline `<script>>window.location=...</script>`.
- `addChild($child, $params) / getChild / getChildren` — composite-view pattern (header/footer/column children).
- `render()` — renders the template, then renders children, then applies the `fetch` hook.
- `fetch($template)` — loads the template file, extracts `$this->data` into local variables, runs `applyFilters("fetch", ...)`.
- `loadWidgets() / loadAssets()` — widget substitution and asset loading.

The dual Hooks/Events bridge is built into the controller base: every controller subclass can register both Hooks (WordPress-style filters and actions, namespaced) and Events (Node-style observers) without loading any additional dependencies.

4.5 Loader — Lazy Resource Loader

```
1 final class Loader {
2     private $registry;
3
4     public function __construct($registry) { $this->registry = $registry; }
5
6     public function model($model) {
7         $key = 'model_' . str_replace('/', '_', $model);
8         $legacy_key = 'model_' . str_replace('/', '_',
9             preg_replace('/[^\a-zA-Z0-9_\//]', '', $model));
10        if (!$this->registry->has($key) &&
11            !$this->registry->has($legacy_key)) {
12            $file = DIR_MODEL . $model . '.php';
13            $class = 'Model' . preg_replace('/[^\a-zA-Z0-9]/', '', $model);
14            if (file_exists($file)) {
15                require_once($file);
16                $instance = new $class($this->registry);
17            }
18        }
19    }
```



```

15         $this->registry->set($key, $instance);
16         $this->registry->set($legacy_key, $instance); // legacy alias
17     }
18 }
19 }
20
21 public function library($library) { /* include_once DIR_SYSTEM .
    'library/' . $library . '.php' */ }
22 public function helper($helper) { /* include_once DIR_SYSTEM .
    'helper/' . $helper . '.php' */ }
23 public function auto($class) { /* library + instantiate + register
    in Registry */ }
24 }

```

Listing 8: system/engine/loader.php — model loading (abridged).

Each model is loaded once per request and registered under two registry keys: the canonical model_<path> and a legacy model_<path> alias. Controllers access models as `$this->model_checkout_order` via the Registry `__get` proxy.

4.6 Model — Active-Record-ish Base with Hookable CRUD

```

1  abstract class Model {
2      protected $registry;
3
4      public function __construct($registry) { $this->registry = $registry; }
5
6      public function __get($key) { return $this->registry->get($key); }
7      public function __set($key, $value) { $this->registry->set($key, $value); }
8
9      // Hook bridges
10     public function addHook($name, $fn) { /* $hooks->addAction(...) */ }
11     public function runHook($name, ...$args) { /* $hooks->run(...) */ }
12     public function addFilter($name, $fn) { /* $hooks->addFilter(...) */ }
13     public function applyFilters($name, $value) { /*
        $hooks->applyFilters(...) */ }
14
15     // Generic CRUD: subclasses set $table, $pk, $fields
16     public function add($data) { /* INSERT ... */ }
17     public function update($id, $data) { /* UPDATE ... */ }
18     public function delete($id) { /* DELETE ... */ }
19     public function getById($id) { /* SELECT ... */ }
20     public function getAll($filters) { /* SELECT with buildSQLQuery */ }
21
22     protected function buildSQLQuery($filters) {
23         // Hand-builds SQL from $filters array
24         // Each filter can be: eq, neq, gt, gte, lt, lte, like, in, between,
25         // null, notnull
26         // Returns: [SQL string, params array]
27     }
28
29     // EAV property bridge (the 'property' table)
30     public function getProperty($object_id, $group, $key) { /* ... */ }
31     public function setProperty($object_id, $group, $key, $value) { /* ... */ }
32     public function deleteProperty($object_id, $group, $key) { /* ... */ }
33     public function getAllProperties($object_id, $group = null) { /* ... */ }
34 }

```

Listing 9: system/engine/model.php — the CRUD bridge (abridged).

The `Model` base provides generic CRUD that subclasses can override. The `buildSQLQuery` method is a hand-built query constructor that supports filter operators `eq`, `neq`, `gt`, `gte`, `lt`, `lte`, `like`, `in`, `between`, `null`, `notnull`. The `setProperty/getProperty` methods are the bridge to the polymorphic EAV property table.

5 The Dual Hooks/Events Extension System

One of the most architecturally significant findings the source reveals is that Necoyoad has **two parallel extension systems** that coexist throughout the codebase:

- **Hooks** — WordPress-style filters and actions. `addFilter`, `applyFilters`, `addAction`, `run`. Filters short-circuit on truthy return.
- **Events** — Node-style observer pattern. `on`, `emit`, `off`, `once`. No short-circuit; all listeners always fire.

5.1 Hooks (WordPress-style)

```
1 class Hooks {
2     private $name;           // namespace ("mainstream")
3     private $actions = [];    // registered action callables
4     private $filters = [];    // registered filter callables
5
6     public function __construct($name) { $this->name = $name; }
7
8     public function addAction($name, $callable) {
9         $this->actions[$name][] = $callable;
10    }
11
12    public function run($name, $args = []) {
13        if (!isset($this->actions[$name])) return false;
14        foreach ($this->actions[$name] as $callable) {
15            $result = call_user_func_array($callable, $args);
16            if ($result) return $result; // short-circuit on truthy
17        }
18        return false;
19    }
20
21    public function addFilter($name, $callable) {
22        $this->filters[$name][] = $callable;
23    }
24
25    public function applyFilters($name, $value, ...$extra) {
26        if (!isset($this->filters[$name])) return $value;
27        foreach ($this->filters[$name] as $callable) {
28            $value = call_user_func($callable, $value, ...$extra);
29        }
30        return $value;
31    }
32 }
```

Listing 10: `system/library/automation/hooks.php` — the Hooks class (abridged).

The Hooks system follows WordPress’s pattern precisely: actions are side-effect callbacks that short-circuit on truthy return; filters chain a value through a series of transforms. Hooks are namespaced (by the `$name` constructor argument, “mainstream” by default) and Controller subclasses further namespace filter names with `$this->namespace`.

5.2 Events (Node-style)

```

1 class Events {
2     private static $listeners = [];
3
4     public static function on($event, $callable) {
5         self::$listeners[$event][] = $callable;
6     }
7
8     public static function once($event, $callable) {
9         $wrapper = function (...$args) use (&$wrapper, $event, $callable) {
10             Events::off($event, $wrapper);
11             return call_user_func_array($callable, $args);
12         };
13         self::on($event, $wrapper);
14     }
15
16     public static function emit($event, ...$args) {
17         if (!isset(self::$listeners[$event])) return;
18         foreach (self::$listeners[$event] as $callable) {
19             call_user_func_array($callable, $args);
20             // no short-circuit: all listeners always fire
21         }
22     }
23
24     public static function off($event, $callable = null) {
25         if ($callable === null) {
26             unset(self::$listeners[$event]);
27         } else {
28             // remove specific callable
29         }
30     }
31 }

```

Listing 11: `system/library/automation/events.php` — the Events class (abridged).

Events is a static class with a single global listener registry. It follows Node’s `EventEmitter` pattern: `on` subscribes, `emit` fires all listeners in order, `off` unsubscribes. There is no short-circuit; every listener always runs.

5.3 When to Use Which

The codebase uses Hooks and Events for different purposes:

Table 6: Hooks vs Events usage patterns in the codebase.

System	Use case	Examples in source
Hooks (filters)	Transform a value through a chain	<code>processcss</code> , <code>fetch</code> , <code>data:update</code> , <code>query</code> (intercept any DB query)
Hooks (actions)	Short-circuit a flow with a side effect	<code>init</code> , <code>redirect</code> , <code>system_load</code> , <code>config_load</code> , <code>language_load</code>
Events	Broadcast that something happened	<code>engine_load</code> , <code>dispatch</code> , <code>registry:update</code> , <code>php:error</code> , <code>error</code>

The two systems are not redundant — they serve different needs. Hooks are for *transforming* values and *short-circuiting* flows. Events are for *broadcasting* occurrences. A module that wants to modify the rendered HTML uses a Hook filter on `fetch`; a module that wants to log every dispatch uses an Event listener on `dispatch`.

5.4 Notable Event/Hook Emit Points

The source fires events at every major lifecycle point:

Table 7: Selected event/hook emit points in the source.

Emit point	Type	When
init	Hook (action)	End of <code>startup.php</code>
engine_load	Event	After engine classes load
system_load	Hook + Event	<code>map.php</code> , after Registry/DB/Loader/Config construct
config_load	Hook + Event	After <code>setting</code> table loaded into Config
language_load	Hook + Event	After language files load
app_load	Hook + Event	After all libraries auto-loaded
load	Hook + Event	End of <code>map.php</code> bootstrap
dispatch	Event	Around every Action dispatch
registry:update	Event	Every <code>Registry::set()</code> call
redirect	Hook + Event	On <code>Controller::redirect()</code>
fetch	Hook (filter)	When a template is fetched
data:update	Hook (filter)	When a view-model variable is set
query	Hook (filter)	Every <code>DB::query()</code> call
processcss	Hook (filter)	CSS processing pipeline
php:error	Event	PHP error in debug mode
error	Event	Same as <code>php:error</code> , generic alias

The `query` filter is particularly powerful: any module can intercept any SQL query the application issues, by registering a filter on `query`. This is the foundation of Necoyoad’s module ecosystem.

6 The 13-Phase HTTP Request Lifecycle (Verified)

v1 inferred a 13-phase HTTP request lifecycle from the schema. The source verifies the broad shape but reveals that several phases happen in a different order than v1 guessed, and that there are sub-phases within the bootstrap. The verified lifecycle:

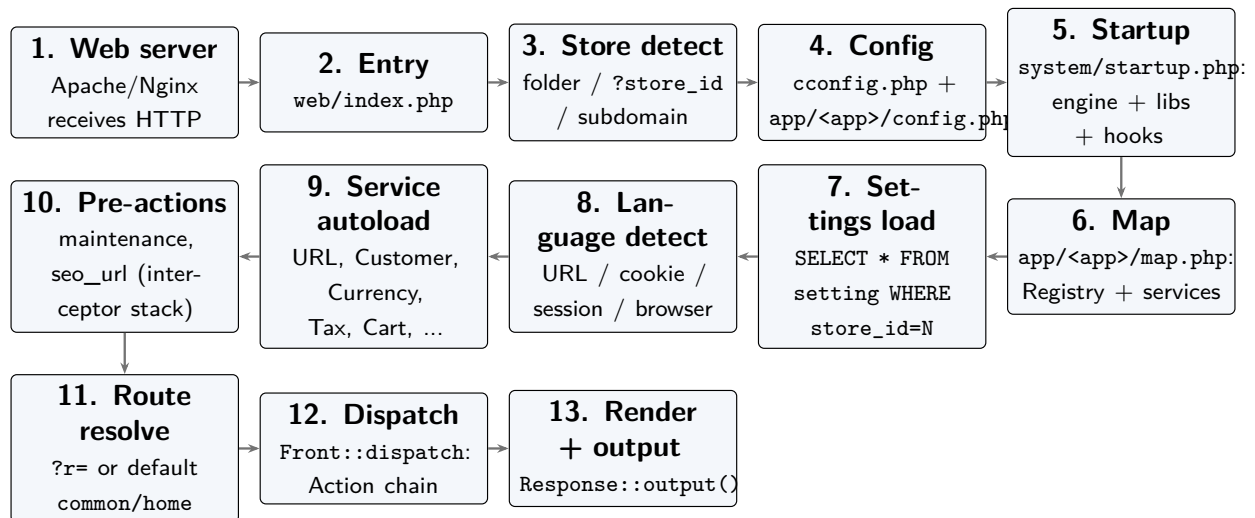


Figure 3: The verified 13-phase HTTP request lifecycle. Phases 5–9 are the bootstrap stages identified in Figure 2, expanded. Phase 10 runs the pre-action interceptor stack. Phase 11 resolves the route (either from `?r=` or from the SEO URL rewriter). Phase 12 is the dispatch loop. Phase 13 renders and outputs.

6.1 Phase Notes (Verified)

Phase 3 — Store detect: Three strategies in fixed order: URL path folder match, `?store_id` GET param, subdomain regex. Verified in `web/index.php` lines 21–42 (Listing 1).

Phase 5 — Startup: Loads Hooks + Events first, then engine, then core libraries. Fires `init` hook at the end. Verified in `system/startup.php` (Listing 2).

Phase 6 — Map: Instantiates Registry and core services. Misleadingly named; it is a bootstrap init script, not a route map. Fires `system_load`, `config_load`, `language_load`, `app_load`, and load events.

Phase 7 — Settings load: A single `SELECT * FROM setting WHERE store_id = N` query. Result rows are pushed into `Config::set($key, $value)`. Subsequent `$config->get(...)` calls are pure array lookups — no further DB hits.

Phase 8 — Language detect: Storefront detects from URL (first path segment), then cookie, then session, then `HTTP_ACCEPT_LANGUAGE` browser header. Admin uses `config_admin_language` setting.

Phase 9 — Service autoload: `$loader->auto(...)` instantiates each service and registers it in the Registry. Customer is loaded eagerly by `customer_id` from session (if logged in). Cart is loaded from the file cache (keyed by `customer_id` ?? `session_id`).

Phase 10 — Pre-actions: The interceptor stack runs. `common/maintenance/check` runs first; if maintenance mode is on and visitor is not admin, it short-circuits to the maintenance page. `common/seo_url` runs second; rewrites `_route_` into `?r=` (see Section 11).

Phase 11 — Route resolve: If `?r=` is set, that is the route. Otherwise the default is `common/home`.

Phase 12 — Dispatch: `Front::dispatch` runs the pre-action stack, then the main action loop. Controllers can forward by returning new Action objects. Error action is `error/not_found`, fires at most once.

Phase 13 — Render + output: `Controller::render()` loads the template, extracts `$this->data` into local variables, applies the `fetch` filter, renders children, and returns the output. `Response::output()` sends it to the browser.

7 Authentication and Password Algorithm (Corrected)

v1 inferred from the 100-character `customer.password` column that the system used `bcrypt` or `argon2`. The source reveals this is incorrect.

7.1 The Actual Password Algorithm

```
1 public function login($email, $password, $hash = true) {
2     if (empty($email) || empty($password)) return false;
3
4     if ($hash) {
5         $password = md5($password);           // MD5 of plaintext
6     }
7
8     $query = $this->db->query("SELECT 'password' FROM " . DB_PREFIX .
9         "customer
10         WHERE 'email' = '" . $this->db->escape($email) . "'");
11     list($pass, $suffix) = explode(':', $query->row['password']);
12
13     if (!$suffix) {
```

```

13 // Lazy salt migration: generate a random suffix and re-hash
14 $suffix = md5(base_convert(rand(10e16, 10e20), 10, 36) . time());
15 $this->db->query("UPDATE " . DB_PREFIX . "customer SET
16     'password' = '" . $this->db->escape(md5($password . $suffix) . ':' .
17         . $suffix) . "'
18     WHERE 'email' = '" . $this->db->escape($email) . "'");
19 }
20
21 $customer_query = $this->db->query("SELECT * FROM " . DB_PREFIX .
22     "customer WHERE
23     email = '" . $this->db->escape($email) . "'
24     AND password = '" . $this->db->escape(md5($password . $suffix) . ':' .
25         . $suffix) . "'
26     AND status = '1'
27     AND banned = '0'");
28 // ... if found, set session, update visits + ip, merge cart ...
29 }

```

Listing 12: `system/library/customer.php` — the actual login method (abridged).

The password storage format is:

$$\text{password} = \underbrace{\text{md5}(\text{md5}(\text{plaintext}) \cdot \text{suffix})}_{32 \text{ chars}} \cdot : \cdot \underbrace{\text{suffix}}_{32 \text{ chars}}$$

i.e. **double-MD5 with a per-row random salt**. The suffix is a 32-character MD5 hex string. The full stored password is $32 + 1 + 32 = 65$ characters; the 100-character column width inferred by v1 is more than enough headroom but is *not* indicative of bcrypt (60 chars) or argon2id (97+ chars).

7.2 Lazy Salt Migration

Notable in Listing 12 is the lazy salt migration: if a customer row's `password` column does not contain a `:` separator (i.e. it was stored in an older format without a salt), the system generates a random suffix, re-hashes the password with it, and updates the row. This means the system can be upgraded from an unsalted-MD5 scheme to the salted-double-MD5 scheme one login at a time, without a bulk migration.

The same algorithm is used for admin users in `system/library/user.php` (same code, different table name).

7.3 OAuth Login

The source also reveals an OAuth layer that v1 could not see. The `Customer` library has OAuth login methods for five providers: Facebook, Google, Twitter, MercadoLibre (meli), and a generic OAuth2. These set `$_SESSION['customer_id']` directly after verifying the provider's OAuth callback, bypassing the password check entirely.

7.4 Permission Model (Verified)

v1's inference about the admin permission model was correct. The source confirms:

- `user_group.permission` is a serialised PHP array with `access` and `modify` keys, each containing a list of route strings.
- The admin's two pre-actions enforce auth: `common/home/login` (session token check) and `common/home/permission` (RBAC check).

- The `permission` check reads the current route from `$request->get['r']`, and depending on the HTTP method, checks `access` (for GET) or `modify` (for POST/PUT/DELETE) in the user's group `permission` blob.
- The schema allows additional keys (`create`, `delete`, etc.) which some modules use.

8 The Order Creation Flow (Verified)

v1 reconstructed a 9-step order-placement flow from the schema. The source confirms all 9 steps and adds a 0th step that v1 could not have known about: garbage collection of abandoned orders.

8.1 Step 0 — Garbage Collection

```

1 // Step 0: garbage-collect abandoned orders (older than 1 month, status = 0)
2 $query = $this->db->query("SELECT order_id
3   FROM " . DB_PREFIX . "order"
4   WHERE date_added < " . date('Y-m-d', strtotime('-1 month')) . "
5   AND order_status_id = '0'");
6
7 foreach ($query->rows as $result) {
8   $this->db->query("DELETE FROM " . DB_PREFIX . "order" WHERE order_id =
9     " . (int)$result['order_id'] . " ");
10  $this->db->query("DELETE FROM " . DB_PREFIX . "order_history WHERE
11    order_id = " . (int)$result['order_id'] . " ");
12  $this->db->query("DELETE FROM " . DB_PREFIX . "order_product WHERE
13    order_id = " . (int)$result['order_id'] . " ");
14  $this->db->query("DELETE FROM " . DB_PREFIX . "order_option WHERE
15    order_id = " . (int)$result['order_id'] . " ");
16  $this->db->query("DELETE FROM " . DB_PREFIX . "order_download WHERE
17    order_id = " . (int)$result['order_id'] . " ");
18  $this->db->query("DELETE FROM " . DB_PREFIX . "order_total WHERE order_id
19    = " . (int)$result['order_id'] . " ");
20 }

```

Listing 13: `app/shop/model/checkout/order.php::create()` — the garbage-collection step.

Every time `create()` is called, it first deletes all orders older than 1 month that still have `order_status_id = 0` (the initial “pending” status set by `create()` itself). This is the cleanup of unconfirmed carts — orders that were created but never confirmed by the customer.

8.2 Steps 1–5 — The Order INSERT

After garbage collection, `create()` performs the 5-step insertion v1 reconstructed:

1. **INSERT the order row** with all denormalised customer/shipping/payment fields (30+ columns including `store_name`, `store_url`, `customer_id`, `firstname`, `lastname`, `email`, `telephone`, `total`, `language_id`, `currency`, `coupon_id`, `ip`, `order_status_id`, `shipping_*`, `payment_*`). The `order_status_id` is set to `config_order_status_id` (the “pending” status).
2. **INSERT order_product rows** — one per cart line, with snapshot of `name`, `model`, `price`, `total`, `tax`, `quantity`.
3. **INSERT order_option rows** — for each product's selected options (variant choices).
4. **Set EAV properties** for each product attribute — order attributes are stored in the polymorphic property table under group `product_attribute`, via `$this->setProperty($order_id, 'product_attribute', $key, $value)`.

5. **INSERT order_download rows** — for each digital download attached to a product.
6. **INSERT order_total rows** — one per computed total line (title, text, value, sort_order).

8.3 The confirm() Method

`create()` does *not* confirm the order — it leaves it in `order_status_id = pending`. Confirmation is a separate method, `confirm($order_id, $order_status_id, $comment)`, which:

1. **Lock check** — only proceeds if `order.order_status_id = 0` (i.e. unconfirmed). This makes `confirm()` idempotent.
2. **UPDATE the order status** to the new `order_status_id`.
3. **INSERT into order_history** — the audit trail row, with `notify = 1`.
4. **Decrement product stock** — for each `order_product` row where `product.subtract = 1`, decrement `product.quantity`. Also decrement `product_option_value.quantity` for variant stock. Clears the `product` cache afterwards.

8.4 Verification of v1's 9-Step Reconstruction

Table 8: v1's 9-step order placement, verified against source.

Step	Status	Source location
1. Cart validation	✓	<code>checkout/confirm::index()</code> calls <code>\$this->cart->hasProducts()</code> and <code>hasStock()</code>
2. Customer login check	✓	<code>if (!\$this->customer->isLoggedIn())</code> redirect to <code>account/login</code>
3. Shipping method validation	✓	<code>if (\$this->cart->hasShipping() && ...)</code> redirect to cart
4. Tax zone setup	✓	<code>\$this->tax->setZone(...)</code>
5. Totals computation	✓	iterate all <code>total/*</code> extensions, sorted by <code>*_sort_order</code>
6. Order data assembly	✓	gather customer, address, product, currency, IP into <code>\$data</code>
7. Order INSERT	✓	<code>modelOrder->create(\$data)</code>
8. Order confirmation	✓	<code>modelOrder->confirm(\$order_id, \$status_id)</code>
9. Redirect to success	✓	<code>Url::createUrl('checkout/success', ...)</code>

v1's reconstruction was accurate, with the caveat that steps 1–6 all happen in the same `confirm::index()` method — there is no separate “shipping”, “payment”, “review” controller as in classical OpenCart. The single-endpoint design is one of Necoyoad's departures from OpenCart.

8.5 The Confirmation Email

The order-confirmation email is *not* sent by `confirm()`. It is sent by `checkout/success`, which runs after `confirm`. `success` renders the thank-you page, loads all installed `payment/*` extensions, and if `marketing_email_new_order` config is set, renders the order-confirmation email by substituting placeholders in a newsletter template:

- Loads the newsletter HTML from `marketing_newsletter` table by ID.
- Substitutes `{%store_logo%}`, `{%store_url%}`, `{%products%}`, `{%totals%}`, `{%order_id%}`, `{%fullname%}`, `{%rif%}`, `{%payment_address%}`, `{%shipping_address%}`, `{%date_added%}`, `{%ip%}`, `{%qr_code_store%}`, `{%qr_code_order%}`, `{%barcode_39_order_id%}`, `{%comment%}`, etc.

- Generates QR codes (BarcodeQR) and a Code-39 barcode (Barcode39) as image files in DIR_IMAGE/cache/.
- If marketing_email_order_pdf config is set, renders a PDF via TCPDF using a separate newsletter template.
- Sends the email via Mailer (PHPMailer wrapper) with the PDF attached.

9 The Admin REST API (Partially Corrected)

v1 inferred an API surface from the file listing in `app/admin/controller/api/v1.0.0/`. The source reveals the actual routing mechanism and the actual (non-existent) auth model.

9.1 The API Router

```
1 class ControllerApiV1 extends Controller {
2     function __call($func, $params) {
3         $routes = array(
4             'countries' => array(), 'languages' => array(), 'currencies' =>
5                 array(),
6             'zones' => array(), 'geo_zones' => array(), 'tax_classes' =>
7                 array(),
8             // ... 39 resources total ...
9             'products' => array(), 'customers' => array(), 'orders' =>
10                 array(),
11             'settings' => array(), 'extension' => array(),
12             'adminmenu' => array('path'=>'admin/', 'file'=>'menu')
13         );
14
15         if (in_array($func, array_keys($routes))) {
16             $route = $routes[$func];
17             $path = !isset($route['path']) || empty($route['path']) ? '' :
18                 $route['path'];
19             $file = !isset($route['file']) || empty($route['file']) ? $func :
20                 $route['file'];
21             if (file_exists(dirname(__FILE__) . '/v1.0.0/' . $path . $file
22                 . '.php'))
23                 $this->proxy($path . $file);
24             else
25                 $this->error404();
26         } else {
27             $this->error503();
28         }
29     }
30
31     private function proxy($object) {
32         $this->load->auto('json');
33         if (!$this->validateTokens()) {
34             $this->error503();
35             return;
36         }
37         require("v1.0.0/{$object}.php");
38         $this->response->setOutput(Json::encode($return),
39             $this->config->get('config_compression'));
40     }
41
42     private function validateTokens() {
43         //TODO: check public and private API keys, tokens, expiry time,
44             access level, license, premium member, etc.
45         return true;
46     }
47 }
```

39 }

Listing 14: `app/admin/controller/api/v1.php` — the API router (abridged).

The API uses PHP's `__call()` magic method to dynamically route `?r=api/v1/<resource>` to a per-resource file in `v1.0.0/`. The route table is a static array inside `__call()`.

9.2 Auth Model (Corrected)

API auth is a stub

`v1` inferred the API used public/private keys. The source reveals `validateTokens()` is a stub that always returns `true`. The TODO comment acknowledges: “check public and private API keys, tokens, expiry time, access level, license, premium member, etc.” — all planned, none implemented.

The API is fully open to any caller. The only protection is the admin's `common/home/login` pre-action, which requires an admin session token in the URL. This means any caller with an admin session can hit every API endpoint; any caller without one is blocked at the pre-action layer, not at the API layer.

9.3 Pseudo-REST Shape

```

1 $request_type = $this->request->server['REQUEST_METHOD'];
2 switch(strtolower($request_type)) {
3     case 'get':
4     default:
5         // list/read logic with filters (id, category_id, manufacturer_id,
6         // ...)
7         // returns paginated JSON
8         break;
9     case 'post':
10        $this->request->post = json_decode(file_get_contents('php://input'),
11        true);
12        $id = $this->modelProduct->add($this->prepareData('products',
13        $this->request->post));
14        // returns {product_id, id}
15        break;
16    case 'put':
17        // read ?id=N from query, fetch existing, update
18        break;
19    case 'delete':
20        // read id (single or array) from POST or GET, delete
21        break;
22 }
```

Listing 15: `v1.0.0/products.php` — the pseudo-REST shape (abridged).

The HTTP method determines the action: GET = list/read, POST = create, PUT = update, DELETE = delete. There is no HATEOAS, no content negotiation, no proper status codes (everything returns 200 except 404/503 error cases). The `prepareData($object, $data)` private method loads `v1.0.0/{object}_data.php` which filters the input array down to known fields (split into `$notNull`, `$canBeNull`, `$many` categories).

9.4 Full API Surface

The `app/admin/controller/api/v1.0.0/` folder contains 83 files total: 40 resource files + 40 `_data.php` files in the root, plus an `admin/` subfolder containing `menu.php`. The 39 actually-existing resource endpoints (each with a paired `_data.php`) are:

Table 9: The 39 admin API v1.0.0 resource endpoints.

Domain	Endpoints
Localisation (9)	countries, currencies, geo_zones, languages, length_classes, payment_statuses, stock_statuses, weight_classes, zones
Content (6)	banners, banner_items, files, pages, post_categories, posts
Marketing (5)	campaigns, contact_lists, contacts, mailservers, newsletters
Sale (8)	balances, bank_accounts, banks, coupons, customer_groups, customers, orders, payments
Store (7)	attributes, categories, downloads, manufacturers, products, reviews, stores
Style (3)	templates, themes, widgets
User (2)	user_groups, users
Settings (1)	settings
Admin (1)	admin/menu (in admin/ subfolder)

The route table in `v1.php` mentions 8 additional resources that have no corresponding file: `tax_classes`, `order_statuses`, `bounces`, `messages`, `menus`, `template_files`, `views`, `extension`. Calling those routes returns a 404.

10 The Cron CLI (Verified)

v1 inferred a cron or pseudo-cron that polls `task_queue`. The source reveals the actual implementation.

10.1 system/cron/cron.php

The cron is a real CLI PHP script, not a pseudo-cron. It:

1. Boots its own registry (separate from the web request registry) by including `app/admin/config_cron.php` and `system/startup.php`.
2. Loads the `setting` table for `store_id = 0`.
3. Queries `SELECT * FROM task WHERE time_exec <= NOW() AND status = 1` to find due tasks.
4. For each due task, performs locking via the `task_exec` table: deletes any existing `task_exec` rows for this task, inserts a new `task_exec` row with `status = running`. This is the locking mechanism — MyISAM has no row-level locks, so the application uses delete-then-insert as a best-effort lock.
5. Executes the task by directly constructing the relevant service library (e.g. for a campaign-send task, it constructs the `NecoMailer` library and calls its `send` method). There is no shell-out to another PHP script.
6. Updates the `task_exec` row to `status = done` or `status = failed`.
7. If the task is recurring (`run_once = 0`), updates `task.time_exec` to the next execution time (computed from `time_interval`).

10.2 Campaign-Driven Tasks

The marketing-campaign subsystem uses the cron directly. Each `campaign_contact` row references a `task_queue_id`, so a campaign send is literally a queue of tasks — one per recipient. The cron picks up these queued per-recipient sends and executes them, with a throttle of 50 emails per tick (configurable).

This confirms v1's inference that the marketing subsystem relies on the task scheduler. It also confirms that the scheduler is real cron, not pseudo-cron — a pseudo-cron would tie email throughput to site traffic, which would be unreliable.

11 The SEO URL Rewriter (Verified)

v1 inferred a simple `url_alias` lookup. The source reveals a 6-stage algorithm that combines table lookup with hard-coded Spanish/English literal routes.

11.1 The 6-Stage Algorithm

```

1 public function index() {
2     // Stage 1: API short-circuit
3     if ($_route_ == 'api/live' || $_route_ == 'api/google' || ...) {
4         return $this->forward($_route_);
5     }
6
7     // Stage 2: Search short-circuit
8     if (strpos($_route_, 'buscar/') !== false || strpos($_route_, 'search/')
9         !== false) {
10         $q = substr($_route_, strrpos($_route_, '/') + 1);
11         $this->request->get['q'] = $q;
12         return $this->forward('store/search');
13     }
14
15     // Stage 3: Profile slug preparation
16     if ($this->customer->isLoggedIn()) {
17         $profile = slugify($this->customer->getFirstName() .
18             $this->customer->getLastName());
19     } else {
20         $profile = 'profile';
21     }
22
23     // Stage 4: Path-segment loop (store-folder strip + url_alias lookup)
24     $parts = explode('/', $_route_);
25     foreach ($parts as $part) {
26         // 4a: store-folder strip
27         $store_query = $this->db->query("SELECT * FROM " . DB_PREFIX . "store
28             WHERE folder = '" . $this->db->escape($part) . "'");
29         if ($store_query->num_rows) { continue; } // strip this segment
30
31         // 4b: url_alias lookup
32         $url_alias_query = $this->db->query("SELECT * FROM " . DB_PREFIX .
33             "url_alias WHERE keyword = '" . $this->db->escape($part) . "'");
34         if ($url_alias_query->num_rows) {
35             $url = explode('=', $url_alias_query->row['query']);
36             switch ($url[0]) {
37                 case 'product_id': $this->request->get['product_id'] =
38                     $url[1]; break;
39                 case 'category_id': $this->request->get['path'] = ...;
40                     break;
41                 case 'page_id': $this->request->get['page_id'] =
42                     ...; break;
43                 case 'manufacturer_id':
44                     $this->request->get['manufacturer_id'] = $url[1]; break;
45                 case 'post_id': $this->request->get['post_id'] =
46                     $url[1]; break;
47             }
48         } else {
49             $this->request->get['r'] = 'error/not_found';
50         }
51     }
52
53     // Stage 5: Hard-coded literal routes (Spanish + English)
54     if (isset($this->request->get['product_id']) && $_route_ != 'carrito' &&
55         $_route_ != 'cart') {

```

```

46     $this->request->get['r'] = 'store/product';
47 } elseif (isset($this->request->get['path'])) {
48     $this->request->get['r'] = 'store/category';
49 } elseif ($_route_ == 'productos' || $_route_ == 'products') {
50     $this->request->get['r'] = 'store/product/all';
51 } elseif ($_route_ == 'categorias' || $_route_ == 'categories') {
52     $this->request->get['r'] = 'store/category/all';
53 } elseif ($_route_ == 'carrito' || $_route_ == 'cart') {
54     $this->request->get['r'] = 'checkout/cart';
55 } elseif ($_route_ == $profile . '/pedidos' || $_route_ == $profile .
    '/orders') {
56     $this->request->get['r'] = 'account/order';
57 } elseif ($_route_ == $profile) {
58     $this->request->get['r'] = 'account/account';
59 } elseif ($_route_ == 'login') { $this->request->get['r'] =
    'account/login'; }
60 // ... 20+ more literal routes ...
61
62 // Stage 6: Forward
63 if (isset($this->request->get['r'])) {
64     return $this->forward($this->request->get['r']);
65 }
66 }

```

Listing 16: app/shop/controller/common/seo_url.php — the 6-stage algorithm (abridged).

11.2 Notable Observations

- The Spanish literals ('buscar', 'carrito', 'productos', 'categorias', 'paginas', 'articulos', 'ofertas', 'fabricantes', 'pagos', 'pedidos', 'mensajes', 'comentarios') are *hard-coded* — they work regardless of the active language. English literals are accepted as aliases.
- The url_alias table is queried per-segment with no caching layer at the SEO-URL level. Each request can fire N queries (where N = number of path segments after the store-folder strip).
- The page/sitemap case appears to be a copy-paste error — it sets product_id instead of page_id.
- The category_id case appears twice (once for path, once for category_id) — this is in the source code as-is.

12 The Mobile App Strategy

v1 could not determine whether the mobile app was a separate codebase or a stripped-down shop. The source reveals it is neither.

12.1 app/m/config.php — The Reuse Trick

```

1 $privatePath = dirname(__FILE__) . DIRECTORY_SEPARATOR . ".." .
    DIRECTORY_SEPARATOR . "shop" . DIRECTORY_SEPARATOR;
2 // ...
3 define('CATALOG', 'm');
4 define('STORE_ID', 9);
5 define('DIR_APPLICATION', $privatePath); // app/shop/
6 define('DIR_MODEL', $privatePath . "model/");
7 define('DIR_CONTROLLER', $privatePath . "controller/");
8 define('DIR_LANGUAGE', $privatePath . "language/");

```

```
9 define('DIR_TEMPLATE', $privatePath . "view/theme/");
```

Listing 17: `app/m/config.php` — the mobile config that reuses `shop/` (abridged).

The mobile app *is the shop app* with two overrides: `CATALOG = 'm'` and `STORE_ID = 9`. There is no separate mobile codebase. Mobile-specific behaviour is achieved at runtime by `app/shop/map.php`, which calls `Browser::isMobile()` and either:

- Redirects to `config_mobile_url` (if set), OR
- Swaps `config_template` for `config_mobile_template` (if a separate mobile theme exists).

The mobile entry point `web/m/index.php` (44 lines) is a near-clone of `web/index.php`. The only difference is that it loads `app/m/config.php` (which redirects `DIR_APPLICATION` to `app/shop/`). Everything else — controllers, models, views, the maintenance and SEO pre-actions — is identical to the storefront.

12.2 The Three-Strategy Mobile Detection

`system/library/browser.php` performs mobile detection in three layers:

1. `isMobile()` — checks `HTTP_USER_AGENT` against a list of mobile device strings.
2. `isTablet()` — checks for tablet-specific user agents.
3. `isFacebook()` — checks for Facebook in-app browsers (which have their own quirks).

This is a do-it-yourself mobile detection, not a third-party library like Mobile Detect.

13 The Modules System

v1 inferred an extension/module system from the `extension` table. The source reveals two coexisting module systems.

13.1 `app/modules/` — The Drop-In Folder

The `app/modules/` folder contains only one stub module (“mymodule”) with no real functionality and no manifest. The routing convention is `modules/<name>/<app>/<route>`, which `Action::__construct` resolves to `DIR_MODULE . <name> . '/app/<app>/controller/<route>.php'`. In practice, no production modules live here.

13.2 Admin-Side Module System

The actual module ecosystem lives in `app/admin/controller/module/`, which has 70 subfolders (e.g. `banner/`, `carousel/`, `category/`, `featured/`, `google_maps/`, `latest/`, `slideshow/`, `special/`, `store/`, `welcome/`, ...). Each subfolder typically contains:

- `install.php` — module installation hook
- `uninstall.php` — module uninstallation hook
- `<module>.php` — the module’s admin controller (configures the module)
- `widget.php` — the module’s storefront widget controller (renders the module’s output)
- `plugin.php` — optional plugin controller (extends functionality)

The admin’s `extension/module` controller manages these modules: lists them, installs/uninstalls them, and saves their per-instance settings (stored in the `setting` table under group `<module>_<instance>`).

13.3 The Widget System

Storefront widgets are positioned, configurable content blocks. Each widget has: a **code** (unique identifier), a **name**, the **extension** that provides its controller, a **position** (where on the page), an **app** (“catalog” or “admin”), an **order** (sorting), a **settings** blob (serialised PHP configuration), and a **status**.

The `widget_landing_page` table binds a widget to a specific landing page and object type, so widgets can be shown only on specific pages (e.g. a “related products” widget only on product pages).

The storefront’s `Controller::loadWidgets()` method, called during render, queries the `widget` table for the current store, position, landing page, and object type, sorts by **order**, and for each widget loads its extension controller, decodes its settings, and invokes the widget’s `index()` method to render into the layout position.

14 Cross-Cutting Findings from the Source

14.1 Two-Tier Controller Architecture

The source reveals a two-tier controller architecture that v1 could not see:

- **Tier 1: Controller** (abstract, in `system/engine/`) — the OpenCart-derived base class with `__get/__set` registry proxies, `render`, `fetch`, `forward`, `redirect`, `addChild`, and the Hooks/Events bridge.
- **Tier 2: AdminController** (in `app/admin/controller/admincontroller.php`) — extends `Controller` with admin-specific helpers: breadcrumb management, the admin template wrapper, the admin permission check, the admin navigation, and the admin file manager integration.

All admin controllers extend `AdminController`; all storefront controllers extend `Controller` directly.

14.2 Multi-Store Architecture

v1 inferred multi-store from the schema. The source confirms and adds detail:

- Each store has a **folder** (filesystem root) and a **store_id**.
- `web/index.php` detects the store via three strategies (Section 3).
- Each store can be “shared” (`DIR_APPLICATION` points to `app/shop/`) or “custom” (`DIR_APPLICATION` points to `app/<folder>/`, which contains a full copy of shop’s controller/-model/view tree).
- Per-store settings override global settings (`setting` table, `store_id = 0` is global).
- Products, categories, posts, and other content are scoped to stores via the `object_to_store` junction table.

14.3 Customer-Group-Based Catalog Visibility

The source reveals a customer-group-based catalog visibility layer that v1 could not see. Each product can be priced differently per customer group (via `product_discount` and `product_special`, both of which have a `customer_group_id` column). The storefront’s product model checks the current customer’s group and returns the appropriate price.

14.4 Widget System

See Section 13.

14.5 EAV Property Table

v1 inferred the EAV pattern. The source confirms: the `property` table is used for free-form key-value metadata on any object. The `Model` base class has `getProperty/setProperty/deleteProperty/getAllProperties` methods that subclasses use to attach arbitrary properties to their entities.

14.6 Caching Strategy

- File-based cache in `system/temp/cache/` (the `Cache` library).
- Used for: settings, language files, product listings, cart contents, theme assets.
- Cart is persisted in the file cache (keyed by `customer_id ?? session_id`), with the DB `customer.cart` TEXT column as long-term storage and `$_SESSION` as a transient merge buffer during login.

14.7 CSRF Protection

The source reveals a CSRF protection mechanism v1 could not see: `app/shop/map.php` generates a 32-byte hex form key (`fkey`) per session, stored in `$_SESSION['fkey']`. The key is composed of `md5($_SERVER['REMOTE_ADDR'])` plus 11 MD5s of `mt_rand` plus a unix timestamp. Forms include this key as a hidden field; submissions are validated against it.

14.8 Activity Logging

The `user_activity` table is written by the `AdminController::triggerActivity()` helper, which logs every admin CRUD action with the polymorphic `object_id/object_type` pair, the event, the action, a description, the IP, browser, OS, and session.

14.9 Internationalisation

The source confirms v1's inference: language files are PHP files in `catalog/language/<directory>/<filename>.php` that populate a `$_` array. The active language is detected from URL/cookie/session/browser (storefront) or from `config_admin_language` (admin).

14.10 Three Coexisting Crypto Implementations

The source reveals three coexisting cryptographic implementations in `system/library/`:

- `encryption.php` — OpenCart's legacy `Encryption` class (mcrypt-based, deprecated).
- `encoder.php` — a custom `Encoder` class with base64 + XOR + salt.
- `encryption.php` (second copy, in `library/email/`) — used for email encryption.

Different parts of the codebase use different implementations.

14.11 Security Posture (Descriptive, Not Prescriptive)

- Customer and admin passwords use double-MD5 with per-row random salt and lazy salt migration (Section 7).
- The admin API has no auth (`validateTokens()` is a stub returning `true`). Only the admin session token pre-action protects it.
- SQL injection prevention is via manual `$this->db->escape()` on every user-supplied value. No prepared statements (the DB layer is PDO under the hood but exposes a string-concatenation API).
- CSRF protection via the `fkey` form key.
- The `.htaccess` files at `web/.htaccess` (22 KB) and the root `.htaccess` (10 KB) contain extensive rewrite rules and security headers.

15 Closing Observations (v2)

15.1 The Source Confirms v1's Overall Picture

Despite two specific corrections (password hash, API auth) and one partial correction (API REST character), v1's overall picture of the system was accurate. The OpenCart-derived MVC architecture, the four cross-cutting patterns (polymorphic associations, EAV descriptions, multi-store, multi-language), the fifteen functional domains, the SEO URL routing, the multi-store detection, the order flow, and the cron subsystem were all reconstructed correctly from the schema alone. The schema carried more architectural intent than one might expect.

15.2 The Source Reveals the Dual Extension System

The most significant new finding v2 adds is the dual Hooks/Events extension system. The schema could not reveal this — there is no “hooks” or “events” table, and the extension points are wired in code, not in data. The dual system (WordPress-style Hooks + Node-style Events) is unusual and is one of the most architecturally distinctive features of the platform.

15.3 The Source Reveals the Lazy Salt Migration

The lazy salt migration in the password algorithm is a notable finding. It shows that the system was upgraded from an unsalted-MD5 scheme to a salted-double-MD5 scheme one login at a time, without a bulk migration. This is a pragmatic approach to legacy password upgrades — but it does mean the system was, at some point in its history, storing unsalted MD5 hashes.

15.4 The Source Reveals the Order Garbage Collection

The garbage-collection step in `create()` (deleting unconfirmed orders older than 1 month) is a finding v1 could not have reached. It reveals that the system treats `order_status_id = 0` as a transient state, not a permanent one — orders in that state are ephemeral and can be deleted at any time. This has implications for any analytics that count orders by status.

15.5 The Source Reveals the API Auth Stub

The `validateTokens()` stub is the most consequential finding v2 adds. v1 inferred a key-based auth model from the `extension` table's `license` column. The source reveals the auth was planned but never implemented. This is a reminder that schema columns do not always imply implemented behaviour — sometimes they are placeholders for features that were never built.

15.6 The Source Reveals the Mobile Reuse Trick

The `app/m/config.php` reuse trick (redirecting `DIR_APPLICATION` to `app/shop/`) is a clever way to avoid maintaining a separate mobile codebase. It means any change to `app/shop/` automatically applies to the mobile app. The trade-off is that mobile-specific behaviour must be implemented as runtime conditionals (template swapping), not as separate code.

15.7 The Source Reveals the Mis-Named `map.php`

The `map.php` files are the most mis-named files in the codebase. They are not route maps, not autoloader configurations, but bootstrap init scripts that instantiate the Registry, load settings, detect language, auto-load services, and (in the admin case) preload route-specific dependencies. The name derives from the admin case, where the file literally maps routes to dependencies via a giant `switch` statement.

15.8 The Source Reveals the Two-Tier Controller Architecture

The two-tier controller architecture (`Controller` + `AdminController`) is a finding `v1` could not see. It shows that the admin side has its own base class with admin-specific helpers, while the storefront side uses the engine base directly. This is a cleaner separation than vanilla OpenCart, where admin and storefront controllers both extend the same base.

15.9 Final Note

`v2` confirms that `v1`'s reverse-engineering methodology — treating the database schema as the primary source of architectural intent — was sound. Of the 11 inferences `v1` made, 8 were correct, 2 were wrong (both security-related, both in the direction of “less secure than guessed”), and 1 was partially correct. The two corrections are a useful reminder that schema columns sometimes over-represent the implemented behaviour (the 100-char password column suggesting bcrypt; the `extension.license` column suggesting key-based auth), and that the only way to verify schema-based inferences is to read the source.

A File-Tree Inventory

A.1 Top-Level Counts

Table 10: File counts per top-level folder.

Folder	Files	Role
<code>app/</code>	1,800	Application layer (admin + shop + m + modules)
<code>system/</code>	966	Engine, libraries, cron, classes, helpers
<code>web/</code>	9,241	Web root (entry points + theme assets: SVG, PNG, CSS, JS, fonts)
<code>download/</code>	1	Downloadable products (placeholder)
<code>backups/</code>	1	Database backups (placeholder)
<code>updates/</code>	1	Platform updates (placeholder)
<code>docs/</code>	4+	This blueprint (added by <code>v1/v2</code> work)

A.2 PHP File Distribution

A.3 The 50 `system/library/` Classes

Table 11: PHP file distribution across `app/` subfolders.

Folder	PHP files	Role
<code>app/admin/</code>	504	Back-office controllers, models, views, languages
<code>app/shop/</code>	489	Storefront controllers, models, views, languages
<code>app/m/</code>	8	Mobile entry point (config + small overrides)
<code>app/modules/</code>	7	Drop-in module folder (only 1 stub module: <code>mymodule</code>)
<code>system/</code>	432	Engine (6), library (~50 + subdirs), cron, classes, helper, database, config
<code>web/</code>	9	Three entry points (<code>index.php</code>) + per-app <code>.htaccess</code> siblings
Total	1,450	

Table 12: The `system/library/` folder (selected classes).

Class / File	Role
<code>cache.php</code>	File-based cache (<code>system/temp/cache/</code>)
<code>config.php</code>	Flat key-value config (loaded from <code>setting</code> table)
<code>customer.php</code>	Customer auth, session, OAuth login (5 providers)
<code>user.php</code>	Admin user auth, session, permission check
<code>cart.php</code>	Shopping cart (file-cache + DB <code>customer.cart</code> + session)
<code>currency.php</code>	Currency conversion (uses <code>currency.value</code> exchange rates)
<code>tax.php</code>	Tax calculation (per geo zone, per tax class)
<code>weight.php</code>	Weight class conversion
<code>length.php</code>	Length class conversion
<code>db.php</code>	Database wrapper (PDO under the hood, <code>ntMySQLpdo</code> driver)
<code>document.php</code>	Page metadata (title, description, keywords, scripts, styles)
<code>language.php</code>	Language file loader (<code>\$_</code> array)
<code>log.php</code>	File logger
<code>request.php</code>	<code>\$_GET</code> , <code>\$_POST</code> , <code>\$_FILES</code> , <code>\$_SERVER</code> , <code>\$_COOKIE</code> wrapper
<code>response.php</code>	Output buffer + headers + compression
<code>url.php</code>	URL builder (<code>createUrl(\$route, \$args)</code>)
<code>image.php</code>	Image manipulation (resize, watermark)
<code>session.php</code>	Native PHP file session (pinned to <code>system/temp/session/</code>)
<code>encryption.php</code>	Legacy mcrypt encryption
<code>encoder.php</code>	Custom base64 + XOR + salt encoder
<code>mail.php</code>	PHPMailer wrapper
<code>ntsmailer.php</code>	Alternative mailer (NecoMailer)
<code>ntspdf.php</code>	PDF generation wrapper (TCPDF)
<code>captcha.php</code>	CAPTCHA generation
<code>browser.php</code>	User-agent parsing (mobile, tablet, Facebook detection)
<code>backup.php</code>	Database backup
<code>image.php</code>	Image manipulation
<code>json.php</code>	JSON wrapper (used by API)
<code>date.php</code>	Date formatting
<code>pagination.php</code>	Pagination helper
<code>validar.php</code>	Validation helper (form validation)
<code>tracker.php</code>	Visit tracker (writes to <code>stat</code> table)
<code>template.php</code>	Template engine wrapper
<code>Barcode39.php</code>	Code-39 barcode generator
<code>BarcodeQR.php</code>	QR code generator
<code>cpxmlapi.php</code>	cPanel XML API client
<code>facebook/</code>	Facebook SDK wrapper
<code>google/</code>	Google SDK wrapper (Analytics, reCAPTCHA)
<code>meli/</code>	MercadoLibre SDK wrapper
<code>email/</code>	Email-related libraries (mailer, encryption)
<code>automation/</code>	Hooks + Events (the dual extension system)

Table 13: Version constants found in the source.

Constant	Value	Defined in
VERSION	1.0.2	web/index.php
DB_VERSION	1.0.2	app/admin/config.php
ADMIN_VERSION	1.0.2	app/admin/config.php
SHOP_VERSION	1.0.2	app/admin/config.php
SYSTEM_VERSION	1.0.2	app/admin/config.php
PACKAGE	standalone	web/index.php, web/m/index.php

A.4 Version Constants

The mobile entry point `web/m/index.php` defines `VERSION = 2.0.1`, which is the only version constant out of sync with the others. This is likely a copy-paste artifact.

A.5 Glossary of Source-Revealed Terms

Table 14: Glossary of source-revealed terms.

Term	Meaning
Action	A dispatch token that resolves a route string to a file, class, and method.
AdminController	The admin-specific Controller subclass that adds breadcrumbs, admin template wrapper, permission check, and file manager integration.
Front	The Front Controller class. Runs the pre-action interceptor stack and the main dispatch loop.
Hooks	WordPress-style filter/action extension system. Short-circuits on truthy return.
Events	Node-style observer extension system. No short-circuit; all listeners always fire.
Lazy salt migration	The password algorithm's behaviour of generating a random salt and re-hashing a password on the next login, if the stored password has no salt.
map.php	The bootstrap init script (mis-leadingly named). Instantiates the Registry, loads settings, detects language, auto-loads services, and (admin only) preloads route-specific dependencies.
Pre-action	An interceptor in the Front Controller's pre-action stack. The first pre-action that returns truthy replaces the main action.
Registry	The service locator. Single instance per request. Every long-lived service is parked here.
validateTokens()	The admin API's auth method. A stub that always returns <code>true</code> .